

Misconfigured or faulty?

Thermal paste was only the first example. The catalogue crosses RAM, storage, the GPU, the power supply, the sensors and the BIOS settings: every failure named, quantified, filed into its family — and its remedy filed at the right tier. Because a capable agent is also an agent capable of critical errors: three barriers, eight rules, zero automatic writes. Think of everything, touch almost nothing.

Ryzen 9 5950X · AIO 240 · ASUS B450

omarchy-firmware · 18 named findings · 4 families · 3 barriers · 8 rules

Contents

1. Ten examples like that: from anecdote to system	0
1.1 The thermal-paste lesson: a method, not a case	1
1.2 The plan of this volume	2
2. The reading grid: four families, four remedies	2
2.1 Why the family determines the remedy	3
2.2 Where the evidence comes from: the sources inventory	5
3. The catalogue, family by family	6
3.1 Memory: the well-behaved unstable	6
3.2 Storage: the counters that confess	6
3.3 GPU: Xid codes, junction and throttle reasons	7
3.4 Power supply and power stages: the rail and the VRM	7
3.5 Motherboard: the battery, and the sensors themselves	7
3.6 The BIOS misconfigurations: the class where the agent may act	8
4. The critical errors: the doctrine	8
4.1 Three error sources, three ramparts	8
4.2 The eight rules	10
4.3 What stays forbidden forever	11
5. What it changes in the base: Phase 3	12
5.1 Four new T0 tools	12
5.2 The first write: the T1 HAL, guarded by the doctrine	12
5.3 Roadmap, updated	13
6. The limits, one more per volume	14
7. Think of everything, touch almost nothing	15
7.1 What a more incredible BIOS finally means	15
7.2 What remains open	15
Sources and references	16

1. Ten examples like that: from anecdote to system

Thermal paste was one example among many: I can give you more than ten like that — components that have a problem either because they are misconfigured or because they are faulty. That is precisely where the BIOS could become far more incredible than an official vendor BIOS. But careful: it also means critical errors become possible. You really have to think of everything.

Those words launched this volume, and they contain the three decisions that structure it. First decision: move beyond the thermal case. Volume 3 proved that missing thermal paste is detectable by sensor correlation; stopping there would be indecent, because the same method applies to the whole machine. An unstable memory stick, a wearing SSD, a GPU falling off the bus, an exhausted CMOS battery, a lying sensor, memory throttled to 2133 MT/s because an XMP profile stayed off:

every example follows the same pattern — scattered sensors, a correlatable signature, a verdict that can be named. Volume 3 built an instrument; this volume applies it everywhere it can look.

Second decision: distinguish the families, because the remedy distinguishes them. A **misconfigured** component is corrected in software, usually in the BIOS, often in thirty seconds, always reversibly. A **faulty** component is repaired with the hands, once the verdict is named and quantified. A **degraded** component only shows up with long-term memory, because every snapshot looks normal. A **firmware bug** is corrected by an update only the vendor can produce. These four families do not differ in severity; they differ in **where the repair happens**: software, hands, time, vendor. Confusing the families means paying for the wrong remedy — applying a fan curve to a dead pump, re-flashing a BIOS for a CMOS battery, re-seating a cooler while a memory stick overshoots its timings.

Third decision, which gives the volume its second title: own the fact that competence creates risk. An agent able to traverse the whole catalogue is also an agent able to make critical errors — flashing an SPI chip with the wrong board's image, pushing a voltage that fatigues the power stages, disabling the thermal protection that would have saved the machine, masking a real failure behind a setting that silences it. The official BIOS is limited and harmless; our super-BIOS must choose to be both at once: limited by construction, not by lack of means. That is the object of chapter 4, which turns the warning into an eight-rule doctrine and into tested barriers in the code.

18	4	3	0
named findings across the whole catalogue	cause families, four places of repair	barriers between a suspicion and a write	automatic writes, no T3 path in the base

1.1 The thermal-paste lesson: a method, not a case

Let us look once more at what volume 3 actually established, because it is a method before being a result. Facing my CPU overheats, the engine did not guess: it read four scattered sensors (the k10temp Tctl, the coolant temperature, the fan headers on the Super I/O, ambient), it computed a physical quantity (the die-to-coolant thermal resistance, 0.29 °C/W healthy against 0.585 °C/W without paste), it observed a dynamic (85 °C reached in 4.0 seconds against 12 to 20 in a healthy state), and it rendered a named verdict — interface-degradee — with its evidence and its confidence level. No step is specific to thermal: read sensors, correlate, compare to a physical threshold, name, doubt when needed. The thermal paste was a clinic's first patient, not the only conceivable one.

The generalisation fits in one question, asked in front of every symptom: **what does the machine already know that it is not saying?** The SSD knows it is wearing out — it keeps a wear-percentage counter and a media-error register, written by its own firmware. The GPU knows when it falls off the bus — the driver journals a dated Xid code. The memory knows when an XMP profile does not hold — the controller re-trains it at the next boot, and the kernel journal records machine errors. The Super I/O knows the 12 V rail

dropped to 11.18 V — it exposes an ADC reading. None of this requires new hardware: nearly the whole catalogue rests on data the kernel already exposes to any patient reader. The rarity was never in the information; it is in the correlation, and in the language that makes it human.

1.2 The plan of this volume

Chapter 2 sets the reading grid: the four cause families, their remedies, and the complete catalogue in two tables — hardware failures on one side, BIOS misconfigurations on the other. Chapter 3 walks the catalogue family by family, with the physics and the evidence of each: memory, storage, GPU, power, board, sensors, then the crown class of settings. Chapter 4 builds the critical-errors doctrine: three error sources, three barriers, eight rules, and the risk matrix that fixes the agent's playing field. Chapter 5 translates it all into the base: four new T0 tools, the first doctrine-guarded T1 write, the updated roadmap. Chapter 6 keeps the honesty tradition — the limits, one more per volume — and chapter 7 places the whole in the trajectory of the four volumes. As always since the first: everything asserted here has been executed, tested or measured on the delivered base, and the quoted thresholds are the code's.

2. The reading grid: four families, four remedies

It is acting up is not a diagnosis; it is the beginning of an investigation. The first thing the investigation produces is not the precise cause but the **family** of the cause — because the family determines the remedy, the place of the remedy, and who must apply it. This section formalises the grid, then walks it across the whole machine: table 2.1 covers hardware failures, table 2.2 covers BIOS misconfigurations. Every row of both tables corresponds to a named finding in the base's vocabulary — the same discipline that took overheating to interface-degradee in volume 3.

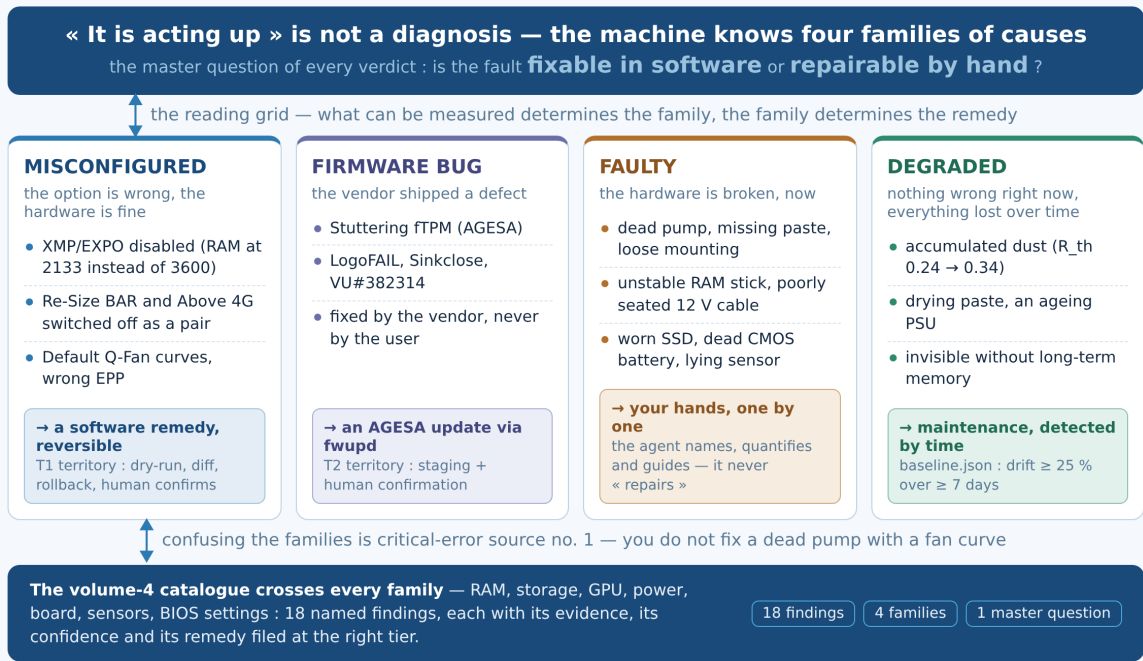


Figure 2.1 — The reading grid: four cause families, four places of repair. The master question in front of every verdict: is the fault fixable in software or reparable by hand?

2.1 Why the family determines the remedy

The distinction is not academic: it is operational all the way to the security tier. A misconfiguration is by definition **reversible** — it can be set and unset, which makes it the only ground where a software write by the agent is conceivable, with dry-run, diff and rollback (the T1 tools of chapter 5). A firmware bug does not belong to the user: it is **reported**, it is corrected by an AGESA update prepared by the vendor, and the base limits itself to staging it behind a human confirmation — the T2 ground. A faulty component is not corrected by software at all: the agent names, quantifies, guides, but the gesture belongs to the hands; any attempt to fix a physical failure by configuration is precisely the archetypal critical error, because it masks the symptom without touching the cause. A degraded component, finally, demands **time** as its sensor: that is volume 3's baseline.json, extended here to every family — long-term drift is evidence in its own right.

Component	Failure (family)	Measurable evidence	Named finding	Filed remedy
Memory	Unstable XMP/EXPO (misconfigured)	freezes under memory load; logged MCE errors (rasdaemon); DRAM re-training at the next boot	memoire-instable	memory profiles: lower the frequency or check voltage (setting)
Memory	faulty stick, tired slot (faulty)	freezes with no software trace; file corruption; recurring MCEs always on the same pattern	memoire-defectueuse	swap sticks, test slot by slot (hands; memtest arbitrates)
Storage	End-of-life NVMe SSD (degraded)	SMART: high percentage_used, dropping available_spare, growing media_errors	nvme-degradation	back up then replace (hands)
Storage	NVMe overheating (misconfigured: no heatsink, slot position)	composite temperature > 80 °C; throttling logged (critical bit of the health log); collapsing throughput	nvme-chaud	heatsink, airflow, possibly a slot swap (hands)
Storage	Degraded PCIe link: riser, contact, extender (faulty)	LnkSta dropping (x4 to x1), AER errors in dmesg, collapsed throughput with clean SMART wear	lien-pcie-degrade	re-seat, replace the riser (hands)
GPU	VRAM overheating (misconfigured: curve, dust)	memory temperature > 95 °C; throttle reasons (SW Thermal Slowdown); performance dips under graphics load	gpu-chaud	fan curve (T1), maintenance, case airflow
GPU	Bus loss: PCIe cable, connector, power (faulty)	dated Xid 79 (fell off the bus) in the journal; resets under load; low 12 V rail at the same time	gpu-alimentation-instable	re-cable, ban splitters, test off the riser (hands)
PSU	12 V rail out of tolerance (degraded)	Super I/O ADC reading < 11.4 V under load (owned $\pm 3\%$); reboots under load with no thermal signature	tension-12v-basse	multimeter measurement then replacement (hands)
Motherboard	VRM overheating (misconfigured: aggressive profile, weak airflow)	VRM sensor > 89 °C while Tctl stays moderate; silent stage throttling	vrn-chaud	a soberer CPU profile (T1), airflow, heatsink (hands)
Motherboard	Depleted CMOS battery (faulty)	RTC clock jumps at boot (wide NTP corrections); UEFI settings lost after a power cut	cmos-pile-faible	replace the CR2032 (hands); owned low confidence
Sensors	A lying instrument (faulty)	value stuck despite load; implausible reading (> 150 °C, negative); gross inconsistency between neighbouring sensors	capteur-suspect	suspend the diagnosis: one does not reason on a dubious eye
Case	Insufficient or inverted airflow (misconfigured)	intakes at 0 rpm; high internal-to-ambient ΔT ; long-term R_th drift (baseline)	refroidissement-insuffisant	curves (T1) then airflow rework (hands)

Table 2.1 — The hardware catalogue: twelve failures the machine already knows about, each row carrying its family, its evidence and its filed remedy.

BIOS setting	Evidence readable from Linux	Named finding	Filed remedy
XMP/EXPO off	dmidecode: Configured Memory Speed at 2133 MT/s on 3600-rated modules	memoire-bridee	enable the profile: T1, measurable before/after gain
Re-Size BAR and Above 4G off	BAR1 total at 256 MB instead of 8192 MB (nvidia-smi); Resizable BAR capability present in lspci	rebar-inactif	enable the pair: T1 confirmed, reboot required
Default Q-Fan curves	pwm_enable and pwm values on hwmon; incoherent noise/setpoint ΔT	courbes-default	fans.curve.set: T1 with dry-run and rollback (P3)
EPP on power on a work machine	energy_performance_preference readable per CPU; degraded average frequencies	profil-sobre	cpu.epp.set: T1, reversible
Unstable fTPM (a bug, not a setting)	freezes correlated with TPM use; AGESA version in the journal (the fw.audit.cve tool)	ftpm-stutter	AGESA update: confirmed T2 staging
Unsuitable boot chain	efibootmgr order and entries; the Limine+UKI chain (vol. 1) mis-positioned	boot-chain	re-order via efibootmgr: T2, backup before write

Table 2.2 — The BIOS misconfigurations: the only class whose remedy is software, hence the only one where the agent acts — guarded by chapter 4's barriers.

Cross-reading the two tables says the essential: eighteen findings, four families, and only six rows where the agent is ever allowed to write. Everything else contents itself with reading — free, journaled, risk-free — and with speech: naming, quantifying, guiding. That deliberate imbalance (six writable rows against eighteen knowable ones) is the first translation of the warning that founded this volume: critical errors are possible; it is therefore decided that the overwhelming majority of the catalogue never opens the hand.

2.2 Where the evidence comes from: the sources inventory

A catalogue without sources would be poetry. Every row of table 2.1 rests on an interface the AM4 + Linux platform already exposes: **k10temp** for Tctl and the die temperature; the ASUS boards' **Nuvoton Super I/O** (nct6775 driver) for fan headers, board thermistors, the VRM and the 12 V rail ADC; the **kernel journals** for machine errors (MCE, collected by rasdaemon), PCIe errors (AER) and the NVIDIA driver's Xid codes; the normalised **NVMe health log** (SMART) for wear, media errors and composite temperature; **drivetemp** for SATA disks; **dmidecode** for the announced memory configuration; **lspci** for the actually negotiated link width and speed; **nvidia-smi** for throttle reasons and VRAM temperature; **efibootmgr** for the boot chain. It is volume 3's principle again: the kernel is already the oracle, the agent is the reader who correlates — and the tables' evidence column is literally the list of files and interfaces consulted.

Two practical consequences follow from this inventory. First, coverage depends on the platform: a board exposing fewer thermistors will lower some findings' confidence, never their honesty — the base then says indeterminate, here is what is missing, as in volume 3. Second, each source has its own cadence: sensors read continuously, journals accumulate, wear counters advance over weeks. The base does not pretend to sample everything forever — volume 3's frugality forbids it — it reads on demand and compares against its memory; that is exactly the role of baseline.json, extended in chapter 5 to every family of the catalogue.

3. The catalogue, family by family

This chapter walks the tables with their physics: for each family, what actually happens in the silicon or the copper, and why the chosen sensors can be trusted. The writing rule is volume 3's: every assertion cites its measurement, and every uncertainty is named instead of airbrushed. The thermal findings (pump, paste, radiator, VRM, 12 V) are not repeated here — volume 3 established and tested them; this chapter covers everything that was still missing.

3.1 Memory: the well-behaved unstable

Faulty memory is the sneakiest failure in the catalogue, because it perfectly imitates a software problem: random freezes, file corruption, application crashes, sometimes clean reboots with no trace. On a desktop AM4 platform without ECC, no hardware counter counts bit errors in real time; the base therefore works with what the platform gives: the **machine check errors (MCE)** the kernel logs and rasdaemon collects, the **DRAM re-trainings** that lengthen a boot after an unstable profile, and the **temporal correlation** between freezes and memory load. The memoire-instable verdict is rendered at medium confidence at best, and the base says so: the final arbiter remains a memtest86+ run overnight, a human action it prepares (key, instructions, duration) without being able to execute. That is an owned limit of chapter 6, not a discovery: the platform imposes it.

The misconfigured side is more generous, and fortunately so: it is the most commonly forgotten setting of personal computing. DDR4-3600 memory left at JEDEC runs at 2133 MT/s — a bandwidth loss of more than forty percent, invisible to the user and perfectly readable: dmidecode announces the **configured** speed next to the modules' **capable** speed. The memoire-bridee finding is therefore of rare reliability: textual evidence, a known remedy (enable the XMP/EXPO profile in the BIOS), a before/after gain demonstrable by the same reading. It is the ideal candidate for Phase 3's first confirmed T1 write — reversible, measurable, and very probably the catalogue's highest benefit-to-risk line of the whole study.

3.2 Storage: the counters that confess

The SSD is the only component in the catalogue that keeps its own medical file: its firmware maintains a normalised health log, readable by smartctl or nvme, and writes there what it knows of its own death. The nvme-degradation finding crosses three indicators: **percentage_used** (the wear computed by the controller), **available_spare** (the remaining reserve blocks — when it drops, the manufacturer itself declares the alarm), and **media_errors** (read errors corrected, then uncorrected). The reading is univocal and the remedy belongs to backups: no setting will regrow NAND. At the other end, the nvme-chaud finding is a classic misconfiguration of B450/B550 boards, where the primary M.2 slot sits next to the GPU: under graphics load, a heatsink-less NVMe crosses 80 °C, engages its throttling (the health log's critical bit), and throughput collapses — evidence by temporal correlation between temperature and the performance drop. The remedy fits in a five-euro heatsink and some airflow.

The PCIe link deserves its row because it betrays the contact family: a badly seated riser, a card re-inserted at an angle, a questionable extender, and the link negotiates degraded — x4 instead of x16, Gen1 instead of Gen4 — or starts emitting AER errors logged in the kernel journal. The lspci reading of link states (LnkSta) is an unambiguous photograph, and comparing it with the capability (LnkCap) makes the diagnosis by itself. It is also the accompanying signature of the gpu-alimentation-unstable finding: a GPU that falls off the bus (Xid 79) almost always leaves a trail of link errors or a low 12 V rail behind it. None of these clues asks more than a reading; no vendor BIOS correlates them.

3.3 GPU: Xid codes, junction and throttle reasons

Under the proprietary NVIDIA driver, the kernel journal records the **Xid codes** — dated internal errors that make an expert dossier on their own: Xid 79 means the GPU left the bus (power, cable, contact); the 13/43 family codes signal graphics context crashes; 94/95 contained or uncontained memory errors. In parallel, nvidia-smi exposes what volume 3 wished for every component: the throttle's **reason**. The Clocks Event Reasons field distinguishes SW Power Slowdown (power limit reached), SW Thermal Slowdown (temperature), HW slowdown — and the VRAM temperature is read separately from the die's; on an RTX 3070 it is the one that crosses the thresholds first in games, usually through dust or a lazy fan curve. The gpu-chaud finding follows directly, with its misconfigured side (curve, maintenance) and its faulty side (repeated Xid 79 → power). Chapter 6's limit applies: these sources belong to the proprietary driver; under nouveau/radeonsi other journals speak, the method stays.

3.4 Power supply and power stages: the rail and the VRM

Volume 3 laid the foundations — the tension-12v-basse finding on the Super I/O's ADC reading, with its owned ± 3 percent and its displayed low confidence, and the vrm-chaud finding on the stage's thermistor. This volume adds the correlating element that makes the differential diagnosis: **the shape of the crash**. An ageing PSU crashes under transient load — instant reboots, no trace, never at idle, often at game launch — while a throttling VRM degrades performance without cutting. The base crosses the three signals (low rail, reboot pattern, temperatures) to separate the two, and honestly names what it cannot see: transient quality demands an oscilloscope, and there is no sensor for that on a consumer board. The catalogue row therefore reads: degraded, medium confidence, human arbitration with a multimeter — exactly volume 3's posture on the 12 V, generalised.

3.5 Motherboard: the battery, and the sensors themselves

The depleted CMOS battery is the catalogue's most modest failure and its most ironic: the machine forgets the time and its settings, and the operating system knows — RTC clock jumps at boot show up as wide NTP corrections, and lost UEFI settings are visible by comparing the configuration before and after a power cut. The signal is weak, the confidence owned as low, the remedy trivial and certain: a two-euro CR2032. The finding is above all a demonstration: the catalogue descends to components nobody thinks of monitoring, so clichéd is the failure once named and so mysterious before.

Deeper: the catalogue monitors **its own eyes**. The capteur-suspect finding is the safeguard that makes the other seventeen credible. A stuck sensor (identical value under load and at rest), an implausible reading (Tctl at 200 °C or negative, a header at 400,000 rpm), a gross inconsistency between neighbouring sensors (a coolant at 90 °C while the die stays at 60 °C) are all reasons to suspend the diagnosis before any reasoning. The rule is that of measuring instruments since always: a dubious eye discredits everything it reports, and it is better to render indeterminate, the instrument is suspect than a brilliant verdict built on a false measurement. It is the doctrine's first barrier, and it is tested in the code like the others.

3.6 The BIOS misconfigurations: the class where the agent may act

There remained the crown class, the one that alone justifies the volume's title: the six rows of table 2.2, the only ones where the BIOS could truly become more incredible than the official BIOS in the literal sense — because the remedy is an option the official BIOS owns but never verifies. The vendor ships dozens of options and does not care: nothing in an AMI BIOS warns that memory has been running at 2133 MT/s for two years, that Re-Size BAR is off while the GPU supports it, that the fan curves are in silent mode on a night-compute machine. The agent, meanwhile, reads the difference between capability and configuration — announced memory speed, measured BAR1, observed pwm_enable, current EPP — and renders a finding with the estimated gain. It is the bridge to Phase 3: these settings are reversible, measurable before/after, and their correction goes through the T1 HAL with dry-run, diff, rollback and human confirmation. The day fw.diag.settings displays your memory gains 41 percent of bandwidth for one reversible setting, the official BIOS, once again, will have said nothing.

4. The critical errors: the doctrine

An agent that sees everything is an agent that can ruin everything. That is the exact counterpart of the enthusiasm that founded this volume, and the study takes it as an architectural constraint, not as an end-of-report warning. The conceivable critical errors are not theoretical: flash an SPI chip with another board model's image and the machine becomes a brick; push a SoC voltage for stability and you silently fatigue the power stages; disable the thermal protection to finish a render and you remove the last hardware safeguard; apply an aggressive fan curve to a dead pump and you build a machine that dies in silence, comfortably cooled everywhere except where it matters. The official BIOS makes none of these mistakes because it does almost nothing; our system owes itself to doing much **and** never making them.

4.1 Three error sources, three ramparts

The doctrine starts by classifying errors, because each family of error is countered at a different stage. The **wrong measurement** first: a lying sensor, a transient reading, a sample taken during a ramp — no downstream intelligence will repair a false input, hence the capteur-suspect finding and barrier A (cross-confirmation: a second sensor, a re-measurement, or a bounded active probe). The **wrong reasoning**

next: correlation is not causation; a high R_{th} says the thermal path is degraded, not why — the base answers with explainable physical rules, named differentials (volume 3 already refused to invent a second paste finding on a dead pump) and a displayed confidence. The **wrong action** finally: the diagnosis was right and the remedy wrong, or applied at the wrong moment — barriers B and C (confidence at threshold, the human in front of the diff) and the full dry-run, rollback, re-measure journey exist for that one. No stage is meant to be infallible; the whole is designed so that an error must cross three independent families of ramparts before reaching the machine.

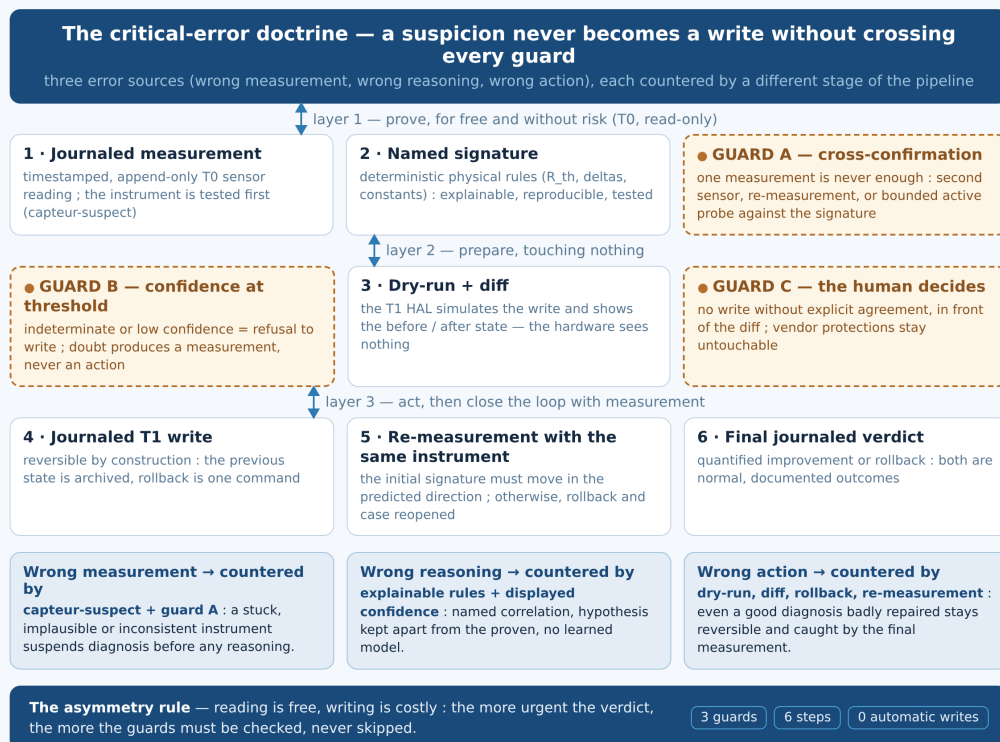


Figure 4.1 — The doctrine's pipeline: a suspicion becomes a write only after barriers A (cross-confirmation), B (confidence at threshold) and C (human agreement), then the loop closes with re-measurement. Each error source is countered by a different stage.

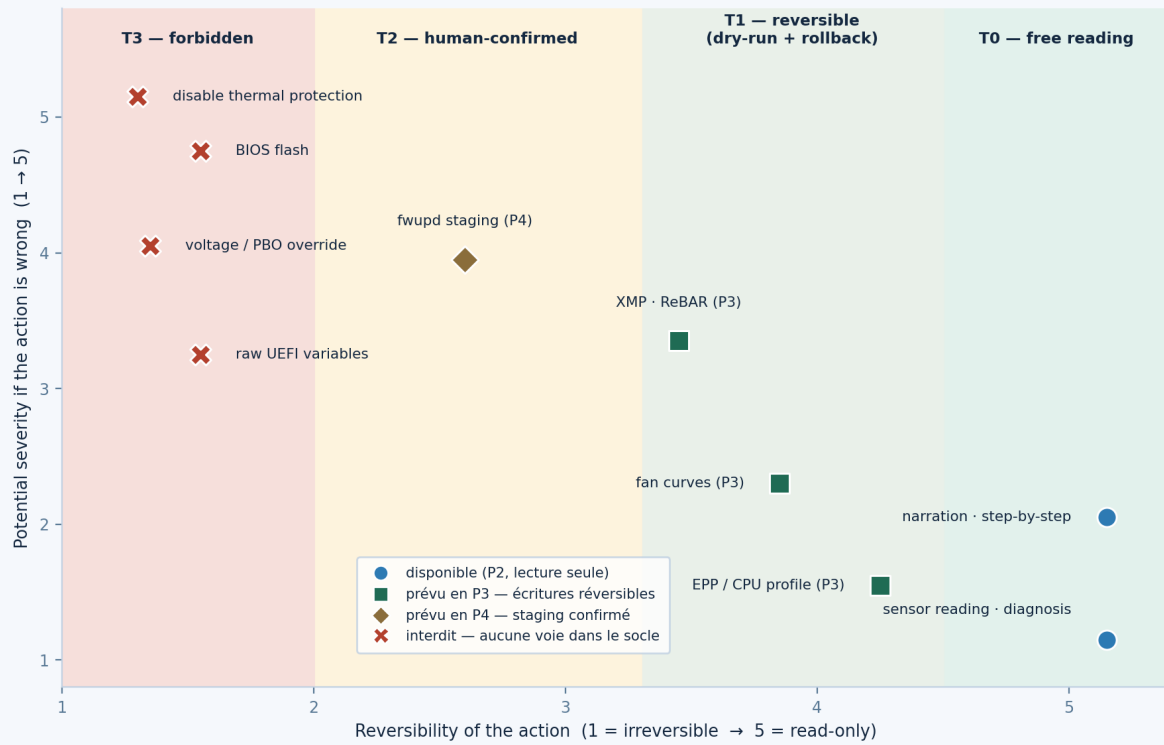


Figure 4.2 — The risk matrix: every possible action of the agent positioned by reversibility and severity. The T0-T3 tiers are exactly the horizontal bands: the playing field narrows as severity rises.

4.2 The eight rules

Rule	What it imposes	Against which error
R1. Reading is free, writing is costly	T0 without friction or confirmation; every write behind a deliberate obstacle (dry-run, diff, agreement)	habit: an agent that writes too much eventually writes badly
R2. No write on a single measurement	the code demands cross-confirmation (second sensor, re-measurement or probe) before any write proposal	wrong measurement
R3. Dry-run mandatory	the T1 HAL simulates and shows the before/after diff; nothing reaches the hardware without having been simulated	wrong action
R4. Rollback by default	every write archives the previous state; going back is a command, not an investigation	wrong action
R5. Vendor protections are untouchable	never disable a throttle, never raise a power or voltage limit, never flash automatically	all of them, and the catastrophes first
R6. Doubt suspends	indeterminate verdict or low confidence: no write possible; the answer is one more measurement	wrong measurement and wrong reasoning
R7. The hands stay human	every physical remedy becomes a dated step-by-step provided by the agent; it never repairs by itself	wrong action (beyond software reach)
R8. The loop closes with measurement	after any action, re-measure with the same instrument: a quantified improvement or a rollback, both normal outcomes	wrong action left unnoticed

Table 4.1 — The doctrine in eight rules: each imposes a code mechanism, not a charter intention.

Two rules deserve a comment because they reverse habits. R2 makes cross-confirmation a condition of code, not of politeness: the write tool literally refuses to open if the carrying verdict has not been corroborated — testable, therefore tested, like volume 2's T1/T2 refusals. R5 draws up the list of doors without handles: thermal protection, power and voltage limits, automatic flashing, raw UEFI variables. There will never be, in any future phase, a nevertheless option for those doors — because the request for the key would one day come at the wrong moment, from a hurried user or an overly persuasive agent. The base has no T3 path by construction; one day that absence will probably be called the study's best architectural decision.

4.3 What stays forbidden forever

Figure 4.2's matrix shows it without detour: the top-left quadrant — severe and irreversible — is not hard to reach; it is **empty of implementation**. BIOS flash, voltage overrides, thermal-protection disabling, raw UEFI variable writes: none of these actions has a function in the base, so no test can call them, so no bug can trigger them. It is a security of a particular kind, security by absence: one does not harden a door, one does not build it. The previous volumes had posed the T1/T2 refusal as a development discipline; this volume raises it to a permanent doctrine — the border between the useful and the dangerous will not move with versions; it will only move if the hardware itself changes nature. The agent explains, the firmware protects, the human decides, and the hands do the rest: the volumes-1-to-3 sentence here completes itself with a final member — **and the T3 tier does not exist**.

5. What it changes in the base: Phase 3

The catalogue is not a poem; it is a development plan. This chapter translates it into omarchy-firmware's evolution: four new T0 tools covering the missing families, then the first doctrine-guarded T1 write — the two bricks that turn a firmware auditor into a machine physician, without ever leaving volume 3's frugality. The project discipline stays that of Phases 1 and 2: every tool enters with its declared tier and its out-of-scope refusal test; the whole check suite stays runnable without hardware, through pre-recorded scenarios.

5.1 Four new T0 tools

Tool (T0, read-only)	What it reads	Findings it feeds
fw.diag.storage	NVMe health log (percentage_used, available_spare, media_errors, temperature), PCIe link states (lspci), AER journals	nvme-degradation, nvme-chaud, lien-pcie-degrade
fw.diag.gpu	Xid codes from the journal, die and VRAM temperatures, Clocks Event Reasons (nvidia-smi)	gpu-chaud, gpu-alimentation-instable
fw.diag.ram	collected MCEs (rasdaemon), DRAM re-trainings at boot, freeze/load correlation	memoire-instable, memoire-defectueuse (medium confidence, memtest arbitrates)
fw.diag.settings	configured memory speed (dmidecode), BAR1 (nvidia-smi), Resizable BAR (lspci), pwm_enable, EPP, efibootmgr chain	memoire-bridee, rebar-inactif, courbes-defaut, profil-sobre, boot-chain

Table 5.1 — Phase 3's four T0 tools: each with its declared tier, its access journal and its write-refusal test.

After Phase 2, the base exposed five T0 tools (audit, boot, CVE, journals, thermal diagnosis); Phase 3 brought them to nine, covering eighteen named findings. Each follows the proven model: a pure function readable by the CLI, a typed MCP tool for Omarchy's harnesses, a dedicated binary when a timer justifies one, and a systematic access journal. The test scenarios grew accordingly: next to the eight thermal scenarios came the worn NVMe, degraded link, repeated Xid 79, recurring MCE, throttled memory and disabled ReBAR fixtures — always the same rule, one scenario names the expected failure and the suite verifies the verdict. And the epilogue belongs to this edition: Phase 3 shipped as 0.3.x — 177 checks, an 11-tool MCP conformance proven, rollbacks verified on both T1 tools — and the catalogue has kept growing since: 12 scenarios, 281 checks, and the whole contract rehearsed nightly on the digital twin.

5.2 The first write: the T1 HAL, guarded by the doctrine

Phase 3 also contains the study's first write: fan curves and the EPP profile move from principle refusal to controlled implementation. The doctrine is not a charter next to the code — it is the code: the tool refuses to run without a prior dry-run; the dry-run produces the before/after diff without touching the hardware; the

real write demands a corroborated carrying verdict (barrier A) and sufficient confidence (barrier B); human agreement is an explicit parameter (barrier C); the previous state is archived and rollback is a command; the final re-measurement journals the outcome. None of these steps is bypassable by a command-line option — the bypass would be a bug, and the refusal tests would catch it. The spirit remains volume 2's: the agent proposes, the HAL disposes, the human decides.

```
omarchy-firmware fans curve --profile quiet --dry-run # before/after diff,
nothing written
omarchy-firmware fans curve --profile quiet --apply # demands a recent dry-run
+ corroborated verdict
omarchy-firmware fans rollback # back to the archived state, one command
omarchy-firmware diag settings --json # throttled memory? ReBAR? curves? EPP?
omarchy-firmware diag storage --json # NVMe wear, temperature, PCIe link
```

5.3 Roadmap, updated

Phase	Content	Testable exit criterion
P1 — done (vol. 2)	4 T0 audit tools + skill + journal	10 state questions without any write (met)
P2 — done (vol. 3)	fw.diag.thermal: signatures, probe, baseline, optional timer	8 scenarios named, 82 tests, MCP conformant (met)
P3 — this volume laid the plan; done (0.3.x)	fw.diag.{storage,gpu,ram,settings} T0 + the T1 HAL (fans, EPP) with dry-run/rollback	18 named findings covered; 100 percent of writes through the HAL with verified diff and rollback; direct paths proven refused
P4 — done (0.4.x)	supervised loop: periodic CVE watch, human-confirmed T2 staging, closure by re-measurement (R8)	12 scenarios, 246 checks; the 5-day no-unconfirmed-write criterion awaits the machine
P5 — done (0.5.x/0.6.x), reshaped	the digital twin: TWIN-1, 28-probe rehearsal, rehearse-diff, capture; trajectory C (AM5 + Dasharo) stays the horizon	rehearsal green on the twin; the same T0 base talks to fixtures as it would to coreboot — continuity by design

Table 5.2 — The roadmap after volume 4, as closed by this edition: every phase through P5 is met.

6. The limits, one more per volume

The tradition holds: every volume publishes what it cannot do, because a diagnosis that does not publish its limits is a sales pitch. The widened catalogue mechanically widens the list — each new family brings its blind spots, almost all imposed by the platform rather than by the base. Table 6.1 lists them with the mitigation actually implemented; the guiding line remains volume 3's: honesty is a feature, a dated indeterminate beats a played certainty.

Limit	Why	What the base does
No ECC on desktop AM4	no real-time hardware counter of memory errors	indirect detection (MCEs, re-trainings, freeze correlation); confidence capped at medium; memtest arbitrates, prepared by the agent
Power transients invisible	no transient sensor on a consumer board; $ADC \pm 3 \%$	diagnosis by shape (reboots under load, rail, temperatures); human multimeter arbitration named as a step
Liquid leaks undetectable	no coolant-presence sensor	owned out of scope; the consequences (rising R_{th}) are, themselves, detected
The pre-OS stays blind	Linux sees neither the POST nor memory training	re-training suspected through the boot duration reported by the human; never quantified by the base
Xid = proprietary driver	under nouveau/radeonsi, other journals, other codes	fw.diag.gpu adapts its sources to the detected driver; the method (journals + temperatures + throttle reasons) stays
CMOS battery: a weak signal	wide NTP corrections have software causes too	owned low confidence; corroboration by UEFI settings loss observed before/after a power cut
Dust vs ambient indistinguishable in a snapshot	an isolated probe cannot tell a warm room from a clogged radiator	memory decides: the long-term baseline (drift) rather than the snapshot; volume 3 made it a finding of its own
Thresholds calibrated 5950X / AM4	the orders of magnitude come from this platform	thresholds cited in the evidence; per-machine baseline for recalibration; an AM5 port planned with trajectory C

Table 6.1 — The widened catalogue's limits, and the real mitigation of each.

A philosophical limit closes the chapter, because it is the true answer to the think of everything that founded this volume: the catalogue covers everything the platform **exposes**, and it will never pretend to cover what the platform hides. Neither the transient of a tired PSU, nor the corrosion of a contact, nor the quality of a solder joint leaves a trace in a sensor; claiming them detectable would be exactly the error the doctrine forbids — believing a measurement says more than it says. The right posture is already in the base: where the platform stays silent, the verdict is called indeterminate, and the agent points to the human and the right tool (multimeter, memtest, eyes) instead of playing the diviner. Think of everything, then, but think accurately: that too is coherence.

7. Think of everything, touch almost nothing

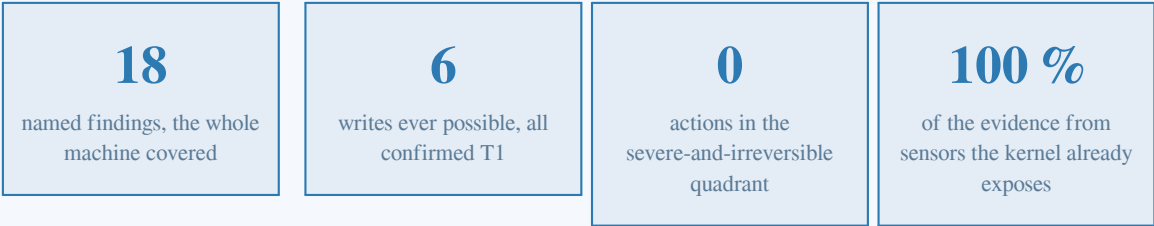
Four volumes moved the front line four times. Volume 1 moved the BIOS's use cases into Linux and surveyed the ground (455 commands, five voids, an omarchy-firmware suite in blueprint form). Volume 2 gave the agent a disciplined grip on the firmware: four T0 tools, locked tiers, a native MCP server. Volume 3 gave it a grip on the thermal hardware: twelve signatures by now, a tested engine, quantified frugality. This volume 4 completes the picture: the grip now covers **the whole machine** — memory, storage, GPU, power, board, sensors, settings — eighteen named findings, four cause families, and above all the doctrine that makes the whole habitable: three barriers, eight rules, an empty quadrant where the irreversible actions will never exist.

7.1 What a more incredible BIOS finally means

The formula that launched the research — an agent that communicates with the BIOS might be ingenious — changed meaning three times across four volumes, and the last one is the right one. It does not mean a bigger BIOS: the SPI flash is saturated, the pre-OS is a hostile environment, and volume 3 quantified why AI will never enter it. It does not mean an agent at the controls: the top-left quadrant of the risk matrix is empty by construction, and that emptiness is what makes the rest possible. It means this: **a machine finally readable from end to end, in which every failure knows its name, its evidence, its family and its remedy — and whose BIOS, intact, keeps the monopoly on the hardware's last word.** The vendor owns the settings but never verifies them; the user suffers the failures but cannot name them; the agent reads everything, understands, explains, and touches almost nothing. That is the exact articulation of the incredible and the safe: omniscience in reading, chastity in writing.

7.2 What remains open

Three works extend this volume without surpassing it. The first was the real execution — and it happened: the base is installed from the tagged release through a checksum-verified installer, an MCP harness can be wired in one line, and the digital twin now rehearses the whole behavioural contract without hardware. The second was Phase 3's coding itself — four T0 tools and the T1 HAL — done as 0.3.x, then hardened through P4's supervised loop and P5's day-0 kit. The third is the catalogue's learning loop: every real failure met on a machine — yours first, then the Omarchy community's — becomes a named test scenario, hence a failure that will surprise no one again. The catalogue is designed to grow by adding rows, never by adding powers: eighteen findings today, twenty-five tomorrow, always zero automatic writes. The overall coherence holds to the end: the study began with a user unhappy with their BIOS and closes with a machine that speaks, explains, and leaves the hands — the only hands — to do what must be done. The machine itself, on 16 September 2026, is the next witness.



Sources and references

NVIDIA. *GPU Xid Errors — driver documentation* (codes 13, 43, 79, 94-95); *nvidia-smi fields: Clocks Event Reasons, BAR1, GPU Memory temperature*. docs.nvidia.com, accessed September 2026.

NVM Express. *NVM Express Base Specification — Health Information Log (SMART/health): percentage_used, available_spare, media_errors, critical_warning*. nvmeexpress.org.

Linux kernel. *Hardware Monitoring: k10temp, nct6775 (Nuvoton Super I/O), drivetemp; the sysfs hwmon interface*. kernel.org documentation.

Linux kernel. *MCE (Machine Check Exception), EDAC/rasdaemon, PCIe AER, dmesg journals; util-linux/lspci: LnkCap/LnkSta — PCI Express link states*.

UEFI Forum. *SMBIOS Reference Specification, type 17 Memory Device (speed / configured speed); efibootmgr* (boot entry management).

Arch Linux wiki. *smartmontools, memtest86+, rasdaemon — diagnostic practices*. wiki.archlinux.org, accessed September 2026.

Brown, DHH et al. *Omarchy — basecamp/omarchy*, commit 5ead870 (4.0.0.alpha): bin/omarchy-agent, agents/skills/, #omarchy:* CLI conventions. github.com/basecamp/omarchy.

Z.ai. *Beyond the BIOS* (vol. 1, 22 p.), *The Agent and the Firmware* (vol. 2, 24 p.), *What the BIOS Cannot See* (vol. 3, 15 p.), September 2026. The CVEs (fTPM, LogoFAIL, Sinkclose, VU#382314), the vendor audit and the thermal physics (R_th, scenarios) are detailed there.

omarchy-firmware. *The bases shipped with volumes 2-3, and their continuations to 0.6.1: lib/firmware_hal/ (tiers.py, diagnostics.py, journal.py, mcp_server.py, storage.py, gpu_diag.py, ram.py, settings.py), tests/test_suite.py* (281 checks as of 0.6.1), tests/fixtures/scenarios/ (12 series). https://github.com/Cheurteenyt/BIOS-